

Effect-bound containment: report authoring pack

13 September 2026

Evidence and planning notes, not a submission manuscript. The sprint’s [Guidelines](#) require the final narrative to be the team’s own writing. Use these checked facts to write in the [official template](#), also saved as `report/official-template.docx`. This pack follows its section order; its PDF is a review copy, not a substitute for the completed template.

Candidate finding-led title: Backend evidence exposes request mutations that upstream logs miss. **Author:** Aditya Pratap Singh. **Affiliation:** confirm before submission; do not imply employer endorsement. **Track:** 1, Containment.

Submission constraints checked on 13 September: main report at most eight pages, excluding references and appendices; required Limitations and Dual-Use Considerations appendix; author names and affiliations. The template suggests a 150–250-word abstract, but the event Guidelines cap the submitted abstract at 150. Use at most 150 for the form and report to satisfy the tighter event rule. Public code and a video are optional.

Abstract planning, not drafted prose: problem (15 words); tested mechanism and environment (30); principal numerical result (45); negative control and scope limit (40); practical takeaway (20). Lead with backend evidence, not a claim of perfect containment. Do not combine different suites into one attack-success percentage.

1. Introduction

Author’s task: explain why an honest upstream log may be an incomplete incident record.

- Incident motivation: Hugging Face’s technical reconstruction describes an intrusion spanning evaluation and third-party systems, using both recovered agent logs and platform logs [1]. It supports the need to compare evidence across boundaries. Our experiments are **not** a replay of its exploits or a demonstration that this prototype would have stopped that incident.
- Tested failure: an approved operation can be substituted before or after an authorization gate. The upstream record can remain accurate about what it saw while failing to describe what the backend received.
- Intended user: an agent-platform or evaluation engineer checking where authorization ends and what evidence an incident reviewer can independently acquire.
- Contributions to describe narrowly: a reproducible boundary-mutation harness; separately scored prevention, detection and utility controls; durable real-twin evidence; and a bounded selective-release experiment with an explicit failure case.
- Do not claim invention of capabilities, complete mediation, simulation or independent logging. The contribution is their measured interaction in this small controlled setting.

Unchanged research question

Can an adaptive effect-bound broker reduce unauthorized agent effects and detect execution mismatches better than intent logging or static authorization, while keeping legitimate actions usable?

Unchanged hypothesis

In a deterministic test environment, server-side capability enforcement combined with dynamic quarantine will prevent more tested unauthorized effects than intent logging or broker-only authorization, at the cost of some additional latency and quarantines.

Hypothesis history is in `hypotheses.md`. H1–H7 were reconstructed retrospectively. H8–H11 were recorded in commit `1378ec7` before the canonical observer comparison, after related earlier examples were already known. This is not a blind or externally preregistered study. Current status is partial support with material qualifications.

2. Related Work

Use these verified sources to explain the comparison, not to claim a comprehensive state-of-the-art benchmark. No head-to-head run against these systems was performed.

Source	Relevant principle	Relationship to this artifact
Saltzer and Schroeder [2]	Complete mediation and fail-safe defaults	Motivate checking at execution and testing paths after the gate.
Hardy [3]	Confused-deputy authority misuse	Motivate scoped authority rather than broad service credentials.
AI Control [4]	Evaluate protocols under intentional subversion	Our attacker is a deterministic mutation script, not a frontier model.
AgentDojo [5]	Extensible agent-security and useful-task evaluation	Motivate benign controls; its tasks and attack distribution are not used here.
CaMeL [6]	Trusted control/data-flow separation and tool-call policy	Closely related capability-based defense; our request comparison does not implement its information-flow guarantees.
Pome [7]	Stateful API simulation with request/state tape	Supplies the actual GitHub twin and backend evidence, not our scorer or capability enforcement.

The older literature notebook includes additional sources not needed for this report. Its Hardy citation has been corrected; do not reuse unchecked bibliographic entries or comparative performance claims merely because they appear in the notebook.

3. Methods

3.1 Threat model and implemented boundaries

- Scripted caller; no LLM inference, training, GPU or production account. Pome uses project-local Node 24.21.0 and CLI 0.43.0, pinned in `package-lock.json`.
- Local capability binds canonical tool/arguments, run ID, request ID, nonce and expiry under HMAC. The server verifies it and checks server policy. Nonces are process-memory state, not durable revocation storage.
- The local server copies mutable input and holds execution/policy locks. Selective release additionally binds a separately signed approval to the exact request, current modeled state and policy revision, with a five-second expiry.
- Three passive evidence views receive the same operator-approved wire request: intent, gate-entry request, or Pome tape. Comparators see method/path/body, not attack labels. Agreement is request consistency, **not semantic safety**.
- The Pome gate is Python code upstream of the REST endpoint. Pome authenticates a bearer credential but does not check our capabilities. Post-gate mutation is an intentional positive attack control, not proof of a Pome vulnerability.
- A spawned observer fetches Pome’s events/state itself, validates tape identity and prefix continuity, writes fsynched snapshots, and returns byte counts and SHA-256 receipts. Retained receipts must be trusted separately from a mutable archive. The observer is a separate process, **not a separate host or trust root**.

Figure 1 specification for the author’s report. Draw caller, intent record, Python gate, Pome server, and separate observer as five boxes. Put mutation point A between intent and gate, and mutation point B between

gate and Pome. Draw the observer’s evidence arrows from Pome’s tape/state endpoints, not from the broker. Label B “detected after execution, not prevented.” Draw all processes inside one shared-host boundary. Show local clone-based quarantine as a separate inset, not as an integrated Pome isolation layer.

3.2 Experiment manifest and denominators

Suite	Unique cells and repetitions	What it establishes
Local matrix	4 configurations $\times$ 10 scenarios $\times$ 20 repeats = 800 cases	Admission/effect outcomes for seven attack and three benign scenarios.
Selective release	$4 \times 9 \times 20 = 720$ cases	Four benign workflows and five deliberately injected backend fault types.
Prior Pome comparison	$4 \times 5 = 20$ fresh twins	Pre/post-gate request substitution against real twin tape/state.
Observer comparison	$2 \times 8 = 16$ fresh twins	Three evidence views on the same cases; eight benign cases, eight attack attempts.
Workflow suite	One controlled run, 11 evidence checkpoints	Named concurrency, revocation, delayed-dispatch and partial-failure schedules.

Twenty repetitions in the local suites are deterministic consistency runs, not twenty independent discoveries. Do not add statistical confidence intervals, significance claims, or estimated production attack rates. In the observer suite, eight attempts produce six executed attacks and two prevented attempts: keep those denominators separate.

Local matrix conditions: intent-only; broker-only; static server with deny-unknown; full server with hold-all quarantine. Attacks are tool/argument substitution, replay, expiry, direct bypass, unknown sensitive read and explicit-label exfiltration. Benign controls are a clean call, registered metadata and supported-but-unregistered status. The weak intent-only condition does not silently retain the server exfiltration guard.

Selective conditions: static deny-unknown, hold-all quarantine, selective release, and an equally informed static policy allowing the same reviewed input shapes. Reviewed operations are `get_status({})` and `repository_exists` on `demo` or `absent`. Simulation rejects modeled writes, sensitive reads, extra output and queued effects. The fifth attack deliberately omits the live effect from the clone. It must stay in the results. Two calls in the repeated-status case are one workflow, not two trials.

Observer conditions: broker-only and full gate. Benign scenarios are metadata, creation, expected 404, and creation followed by injected response loss. Attacks are pre-gate tool/argument substitutions and post-gate creation/file-read substitutions. The read returns Pome’s seeded README outside approved metadata scope. It contains no real secret and changes no final state.

3.3 Scoring, failures and timing

- Local attack success is scenario-defined: a state change, sensitive read, outgoing notification, or unauthorized admission for replay/expiry. These outcomes are not all state-changing damage. Baseline benign completion is an acceptance proxy; its metadata stub does not demonstrate real application usefulness.
- Selective benign completion checks expected returned values; malicious outcomes come from executed records and final state after flushing modeled pending effects.
- Observer attack success checks actual created resources or returned file content, independently of the detector’s alert. Legitimate completion checks the backend objective. An expected 404 can complete a task; response loss can leave a committed task with a client timeout. Neither implies a successfully delivered response.
- Missing/damaged evidence becomes **unknown**, not clean, prevented, detected or missed. A gate alert on a blocked attempt is reported separately from executed detection.
- Dispatch uses `perf_counter`. Backend effect time lies somewhere between dispatch start and completion; subtract both from comparison completion to obtain a delay interval. This is post-dispatch acquisition, not a continuous alerting service.

4. Results

4.1 Local prevention and utility

Table 1. Local matrix totals. AS = attack successes; LC = legitimate completions; FR = false rejection; FQ = false quarantine. Each configuration has 140 attack and 60 benign cases, from seven and three unique scenarios repeated twenty times.

Configuration	AS / 140	LC / 60	FR	FQ
Intent only	140	60	0	0
Broker only	100	40	0	20
Static server	0	40	20	0
Full, hold-all	0	40	0	20

Evidence: `results/matrix_raw.json`, `results/matrix_summary.json`; runner `run_matrix.py`. Interpretation boundary: server enforcement improves the tested outcomes against weaker baselines. Hold-all quarantine adds no demonstrated prevention or utility advantage over static deny-unknown in this matrix (H2).

4.2 Selective-release tradeoff

Table 2. Local selective-release totals. Each configuration has 100 attack cases and 80 benign workflows. Fault scenarios intentionally violate declared harmless behavior.

Configuration	AS / 100	LC / 80	FR	FQ
Static deny-unknown	0	0	80	0
Hold-all	0	0	0	80
Selective	20	80	0	0
Static, same reviewed shapes	100	80	0	0

Evidence: `results/selective-release-v1/raw.json` and `summary.json`; runner `run_selective.py`. Selective release holds four of five modeled fault classes, but all twenty simulation-gap repetitions succeed. Its local comparator flags those twenty after execution. The result is a fidelity-dependent utility tradeoff, not evidence that simulation is safer than denying every unknown action (H6–H7).

4.3 Independent backend evidence

Table 3. Observer comparison, six executed attacks and eight benign controls. Two further attack attempts were prevented. Counts pool two defense configurations for the evidence-view comparison; they are not six independent mutation families.

Evidence view	Detected / 6	Missed / 6	Blocked attempts	
			alerted / 2	False alarms / 8
Intent	0	6	0	0
Gate	2	4	2	0
Backend tape	6	0	0	0

Evidence: `results/observer-comparison-v1/raw.json`, `summary.json` and per-cell snapshot receipts; runner `run_observers.py`. Broker-only permits 4/4 attempts; full enforcement permits 2/4, both post-gate. Both complete 4/4 benign backend tasks. All captures are available; evidence-failure behavior comes from injected regression tests, not naturally observed outages in this matrix.

Specific checks for the report: both post-gate reads return content without state change; gate and intent match the approved request while backend tape does not. Both expected-404 and lost-response controls produce no mismatch alarms. This supports H8–H10 only on the stated cells; 0/8 false alarms is not a reliable population estimate.

4.4 Costs and stress checks

- Local clean-request medians: intent 0.020 ms, broker 0.048 ms, static/full 0.057 ms each. Source `results/resource_summary.json`: five warmups and 100 measurements per condition. Costs exclude real sandbox startup and do not estimate deployment.
- Selective status median: approximately 0.123 ms, versus 0.043 ms hold-all, 0.009 ms static denial and 0.052 ms equally informed static permission. Twenty short local measurements per condition; not production latency or independent safety samples.
- Observer final capture median: 4.99 ms. Six executed-attack delay intervals have lower endpoints 4.47–9.87 ms and upper endpoints 12.71–22.17 ms. Report these as ranges of interval endpoints, **not** a confidence interval or exact detection time.
- Eleven workflow checkpoints include eight-way concurrent replay (one backend call), eight distinct concurrent requests, policy revocation before execution, delayed dispatch, duplicate-create failure, committed write with lost response, rejected replay after response loss, and cross-run denial. Scheduling is harness-controlled, not real CI/webhook behavior or exhaustive interleaving exploration.
- Verified repository checks at preparation: 78 tests pass, including successful attack negative controls. `verify_results.py` validates receipts and recomputes summaries; test success must not be described as every attack being blocked.

5. Discussion and Limitations

Author’s task: connect the results to an auditable engineering requirement, then state the assumptions under which it stops helping. Suggested claims to defend:

- Request authorization and effect observation answer different questions. Neither an accurate intent record nor a signed capability proves what a later backend did.
- The strongest real-twin result is improved mismatch evidence, not containment. The strongest local prevention result comes from execution-time enforcement.
- An equally informed static baseline matters: added knowledge about safe tool shapes must not be mistaken for a simulation benefit. The simulation-gap control makes the remaining dependency on a trustworthy effect model visible.
- Overall hypothesis remains qualified. No single end-to-end, isolated, Pome-backed adaptive deployment has been built or tested. Do not splice the local and Pome experiments together into evidence that such a deployment exists.

One-month continuation, proposed not measured: independently reproduce the frozen suite first; move capability/policy validation into an execution endpoint; repeat the post-gate controls; then add a separately seeded isolated simulator and an observer on a separate trust boundary. Test a held-out tool family and adaptive multi-step attacks with the same utility/evidence-failure denominators. Success would be preservation of legitimate completion while reducing held-out unauthorized effects, not merely more tests passing. Keep H1–H11 unchanged; register new hypotheses before those runs.

6. Conclusion planning

Write one short paragraph in your own words using three claims: measured backend evidence advantage, conditional local enforcement/utility tradeoff, and the failure of upstream enforcement or unfaithful simulation to guarantee safe effects. Do not claim 100% safety, formal verification, state-of-the-art performance, or deployment.

Code and Data

No remote is configured in this checkout. Do not insert an invented repository URL. Provide a reviewer-accessible artifact location or arrange an appendix/package route before submission. No files have been publicly published by this preparation step.

Report material	Reproduction or evidence
Local matrix	<code>run_matrix.py</code> ; <code>results/matrix_raw.json</code>
Selective release	<code>run_selective.py</code> ; <code>results/selective-release-v1/</code>
Real-twin observation	<code>run_observers.py</code> ; <code>results/observer-comparison-v1/</code>
Workflow stress checks	<code>run_workflows.py</code> ; <code>results/pome-workflows-v1/</code>
Full receipt/summary check	<code>.venv/bin/python verify_results.py</code>
Regression checks	<code>.venv/bin/python -m pytest -q</code>

From the repository root, use README.md’s pinned setup. Do not rerun `run_matrix.py` over the frozen evidence merely to read results: its default overwrites its output. The Pome/selective/observer runners require a fresh output directory. Historical interrupted `pome-observed-v2` is not a completed experiment and is excluded.

Author Contributions and LLM Usage notes

Facts to disclose accurately, not a ready-to-submit authorship declaration:

- Aditya supplied the research direction, challenged claims and failure modes, and requested the implementation and evaluation sequence.
- AI coding assistance produced substantial implementation, tests, experiment orchestration, analysis notes and this authoring pack.
- Automated tests and source checks were run in the shared workspace. This is **not** independent human verification. Do not state that all findings were independently verified or the manuscript reviewed by the author until that has actually happened.
- You must write/review the final narrative, confirm authorship and affiliation, and describe the actual AI assistance. Do not claim that this pack is your own writing.

References

1. Larcher, H., Carreira, A., raphael g., and Rannou, C. (2026). *Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident*. Hugging Face, 27 July. [Primary account](#). Author display names as listed on the page. Used for motivation, not our experiment’s outcome.
2. Saltzer, J. H., and Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. Proceedings of the IEEE, 63(9), 1278–1308. [Paper](#).
3. Hardy, N. (1988). *The Confused Deputy*. ACM SIGOPS Operating Systems Review. [DOI](#). Bibliography checked against Crossref.
4. Greenblatt, R., Shlegeris, B., Sachan, K., and Roger, F. (2023). *AI Control: Improving Safety Despite Intentional Subversion*. arXiv:2312.06942. [Paper](#).
5. Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. arXiv:2406.13352. [Paper](#).
6. Debenedetti, E., et al. (2025). *Defeating Prompt Injections by Design*. arXiv:2503.18813, v2. [Paper](#).
7. Pome contributors (accessed 13 September 2026). *Pome Digital Twins*. [Repository](#). Experiments pin npm @pome-sh/cli 0.43.0; they do not run an unpinned current GitHub checkout.

Appendix A. Limitations and Dual-Use Considerations

Required appendix planning notes. Expand or edit into the author’s own analysis.

Security limits: process-memory nonce storage; trusted signing keys and policy; one-host observation; no network sole-egress enforcement; no arbitrary code isolation for the in-process clone; no independent backend-source attestation; no complete model-context taint tracking, covert-channel coverage or downstream-effects model. An authorized secret read followed by an unlabelled output is a known local leak. An initially fabricated, structurally consistent source record can pass validation. Receipts protect against later archive changes only with a trusted retained anchor.

Method limits: deterministic constructed cases, small scenario diversity, shared implementation assumptions, no adaptive LLM attacker, no independent evaluator or external replication. Baseline utility sometimes measures acceptance rather than application output. Pome models selected GitHub routes; simulation fidelity cannot be inferred for arbitrary APIs, production dependencies or live infrastructure. Tape and state are separate requests at quiescent checkpoints, not an atomic global snapshot. Delayed effects are explicitly scheduled fixtures, not real webhooks.

Dual use: mutation hooks can illustrate monitoring blind spots. These experiments use loopback twins, dummy content, temporary credentials and synthetic state; they do not reproduce the July intrusion, include live victim secrets, or test third-party targets. Keep future red-team work in authorized environments. Review disclosure before publishing new exploit or installation mechanisms. Raw logs can contain sensitive arguments in real deployments; redaction, access control and retention must be designed without erasing the evidence needed for comparison. Current Pome Authorization headers in archived tape are redacted.

Publication decisions still needed: author’s factual review; confirmed affiliation; final narrative and AI-use statement; artifact accessibility; appropriate license and any disclosure review. A local commit is not publication or independent validation.

Appendix B. Proposed evidence-bound containment checklist

Draft requirements for engineering review, **not an adopted or certified standard**. A third party can reproduce the benchmark checks without a lab network, but cannot infer a real lab’s isolation, policy quality or compliance from those checks alone.

Proposed requirement	Evidence a reviewer should request	Current status
Bind execution to exact authority and policy	Mutated argument, scope, expiry, replay and revocation cases	Local server implemented; Pome endpoint lacks capability check.
No path around the execution gate	Network/credential boundary evidence and bypass attempts	Network sole-route guarantee not implemented.
Log received/executed requests, including reads	Correlated backend tape plus a read without state mutation	Implemented on tested Pome routes.
Retain independently acquired evidence	Acquisition identity, durable receipts, missing/tampered-evidence checks	Separate process implemented; host/source trust remains.
Treat missing evidence as unresolved	Coverage failures distinct from clean outcomes	Tested unknown state; no operational escalation service.
Revalidate selective-release preconditions	Exact request/state/policy scope and replay tests	Local curated reads only; simulator-mismatch attack succeeds.
Preserve useful work and honest outcomes	Benign tasks, failures, timeouts and false-quarantine counts	Tested narrowly; no general agent-task guarantee.

Appendix C. Final author checklist

- Write the narrative and an abstract of at most 150 words in the official template; remove its guidance.
- Preserve the experiment distinctions, denominators and negative controls above.
- Add Figure 1 from the specified trust boundaries; number figures and tables.

- Confirm author name, affiliation, contribution and accurate LLM-use statement.
- Supply an accessible code/evidence artifact; a path on this machine is not a public link.
- Check actual final PDF pagination: at most eight main pages excluding references/appendices.
- Include the required Limitations and Dual-Use Considerations appendix.
- Verify cited primary sources, rerun checks, and review for sensitive material.
- Submit through the official form yourself or explicitly authorize submission later.